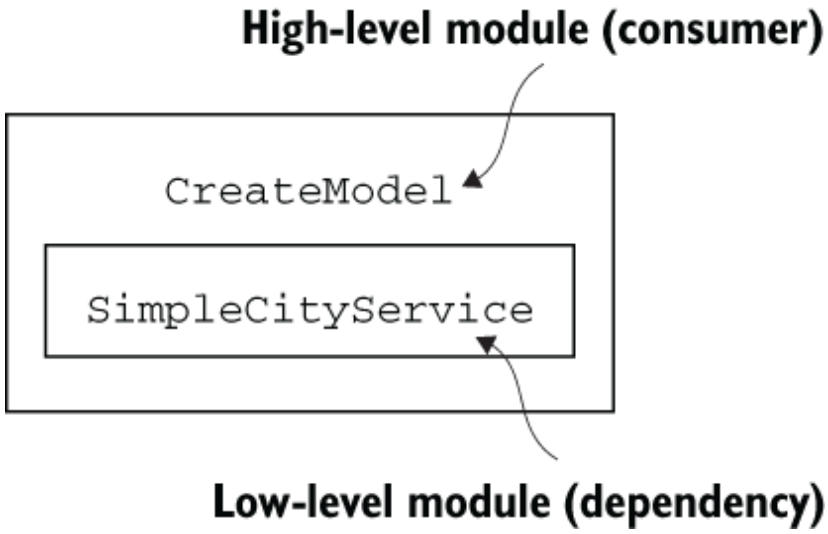



**Figure 7.2 The `SimpleCityServices` is glued (tightly coupled) to the `Create-Model` class.**

You are in breach of a software engineering principle here, the *explicit dependencies principle*, which states that "methods and classes should explicitly require (typically through method parameters or constructor parameters) any collaborating objects they need in order to function correctly" (http://mng.bz/9Vzj). Your collaborating object, the `SimpleCityService`, is an *implicit* dependency of the `CreateModel` class because it is only apparent that the `CreateModel` class depends on the `SimpleCityService` when you look at the source code of the consuming class. Implicit dependencies are to be avoided. They are difficult to test and make the consumer (the `CreateModel`) more brittle and resistant to change.

If you want to change the implementation of the data provider to another one, say that gets data from a database, you will have to go through all the places in the code where you call `new SimpleCityService()` and change it to reference your alternative implementation. And you *will* be changing the implementation in the next chapter. You might think using your